Block4 Token (B4T): whitepaper académico

Nov 4, 2026 · @David Cuadros

Resumen

Block4 Token (B4T) es un token ERC-20 de suministro fijo, creado solo con fines educativos para recorrer el ciclo completo de un activo digital en una red pública. Un contrato mínimo emite 1 000 000 B4T una sola vez; se prueba primero en Polygon Amoy y se despliega una única vez en Polygon PoS, desde una billetera exclusiva del proyecto.

Cuatro integrantes del equipo reciben 250 000 B4T cada uno y ejecutan transferencias y delegaciones que cualquiera puede verificar en Polygonscan. B4T no se vende, no se lista, no ofrece liquidez y no representa ningún derecho económico.

Este documento fija el propósito, las reglas del token, la arquitectura del contrato, el plan de pruebas, el manejo de la billetera del proyecto y los riesgos asumidos. La evidencia del experimento será únicamente información pública de la blockchain.

Equipo: David Alberto Cuadros Mariño, Maximiliano Bustamante Garcia, Valeria Francis Hornung Urrego y Leidy Carolina Obando Figueroa. **Supervisor académico:** Sergio Steven Ramirez Rico. **Institución:** Universidad EAFIT. **Estado:** versión preliminar 0.1, previa a las pruebas en Amoy.

1. Propósito, alcance y exclusiones

El propósito de B4T es que el equipo observe, con evidencia pública, cómo se define, prueba, despliega y usa un token fungible sin introducir incentivos financieros. Entender una blockchain solo desde la teoría no muestra sus restricciones prácticas: costo de gas, manejo de llaves, irreversibilidad de las transacciones y riesgos de seguridad. El experimento las vuelve observables.

Alcance

- Diseño de un contrato ERC-20 de suministro fijo basado en OpenZeppelin.
- Plan de pruebas en la red de pruebas Polygon Amoy (casos A01 a A18).
- Un único despliegue en Polygon PoS, con el código verificado en Polygonscan.
- Distribución inicial y transacciones entre las billeteras de los cuatro integrantes.
- Conciliación de saldos y costos, e informe con evidencia verificable.

Exclusiones

B4T no tiene preventa, cotización, liquidez ni mercado secundario. No se canjea por moneda fiduciaria ni por otros activos, no capta recursos, no custodia fondos de terceros y no se integra en ninguna aplicación comercial. Nada de este documento constituye asesoría financiera ni promesa de valorización.

2. Definiciones

Estos son los términos técnicos que usa el documento y lo que significan para B4T.

Término	Definición
Token fungible	Activo digital cuyas unidades son intercambiables: un B4T equivale a cualquier otro B4T.
ERC-20	Estándar de Ethereum que define la interfaz común de los tokens fungibles: saldos, transferencias, autorizaciones de gasto y eventos [1].
Billetera	Cuenta controlada por una clave privada. En este proyecto es de autocustodia: quien firma controla sus fondos.
Clave privada y frase semilla	Secretos que dan control total e irreversible sobre la billetera. Nunca se comparten ni se suben al repositorio.
Dirección pública	Identificador de una cuenta, derivado de su clave pública. Sirve para recibir tokens y puede mostrarse sin riesgo.
Contrato inteligente	Programa almacenado en la blockchain que ejecuta reglas sin intermediario. Una vez desplegado no puede modificarse.
Gas y POL	El gas mide el trabajo computacional de una transacción. En Polygon PoS se paga en POL, el token nativo de la red [3].

Término	Definición
Chain ID	Número que identifica la red y forma parte de la firma de cada transacción: 137 en Polygon PoS y 80002 en Amoy [3], [9].
Polygon Amoy	Red de pruebas pública de Polygon PoS. Su POL se obtiene en faucets y no tiene valor económico [3].
Hash de transacción	Identificador único de una transacción; con él cualquiera puede consultarla en Polygonscan.
Evento	Registro que el contrato deja en la blockchain: `Transfer` en cada movimiento y `Approval` en cada autorización.
Polygonscan	Explorador público de bloques. Permite verificar que el código publicado coincide con el desplegado.

3. Especificación de B4T

B4T es un token ERC-20 de 1 000 000 unidades visibles con 18 decimales, es decir, 10^24 unidades mínimas, emitido una sola vez al desplegar el contrato.

Elemento	Valor
Nombre	Block4 Token
Símbolo	B4T
Estándar	ERC-20, implementación de OpenZeppelin Contracts 5.6.1
Suministro	1 000 000 B4T, emitidos una sola vez
Precisión	18 decimales: 1 B4T = 10^18 unidades mínimas
Suministro en unidades mínimas	10^24
Titular inicial	Cuenta que despliega: billetera de pruebas en Amoy y billetera exclusiva del proyecto en Polygon PoS

Elemento	Valor
Redes	Polygon Amoy (chain ID 80002) como ensayo; Polygon PoS (chain ID 137) para el despliegue único
Compilador	Solidity 0.8.24, versión fija
Optimización	Activada, 200 ejecuciones
Versión de EVM	cancun
Constructor	Sin parámetros
Licencia	MIT

La billetera del proyecto recibe los 1 000 000 B4T al desplegar y los reparte en partes iguales mediante cuatro transferencias ordinarias. Al terminar, su saldo de B4T es cero.

Cuenta	Saldo tras la emisión (B4T)	Saldo tras la distribución (B4T)
Billetera del proyecto	1 000 000	0
David Alberto Cuadros Mariño	0	250 000
Maximiliano Bustamante Garcia	0	250 000
Valeria Francis Hornung Urrego	0	250 000
Leidy Carolina Obando Figueroa	0	250 000
Total	1 000 000	1 000 000

La cantidad es didáctica: no representa precio, patrimonio ni derecho económico. Las cantidades se firman en unidades mínimas; por ejemplo, 1,5 B4T se envía como 1 500 000 000 000 000 000 unidades. La suma de los saldos debe coincidir siempre con `totalSupply()`.

4. Arquitectura del contrato

El contrato `B4T` hereda de `ERC20` de OpenZeppelin y solo añade un constructor que emite el suministro una vez, por lo que no contiene lógica propia de saldos ni de permisos.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import {ERC20} from "@openzeppelin/contracts@5.6.1/token/ERC20/ERC20.sol";

contract B4T is ERC20 {
    uint256 public constant INITIAL_SUPPLY = 1_000_000;

    constructor() ERC20("Block4 Token", "B4T") {
        _mint(msg.sender, INITIAL_SUPPLY * 10 ** decimals());
    }
}
```

El constructor no recibe parámetros: el nombre, el símbolo y el suministro quedan fijos en el código verificado, y cualquier lector los comprueba leyendo la fuente. `INITIAL_SUPPLY` es una constante de solo lectura que expresa el suministro en unidades enteras; no modifica ningún saldo.

Funciones incluidas

Función	Tipo	Descripción
`name()`	Consulta	Devuelve "Block4 Token".
`symbol()`	Consulta	Devuelve "B4T".
`decimals()`	Consulta	Devuelve 18.
`totalSupply()`	Consulta	Suministro total: 1 000 000 × 10^18 unidades mínimas.
`INITIAL_SUPPLY()`	Consulta	Suministro en unidades enteras: 1 000 000.
`balanceOf(cuenta)`	Consulta	Saldo de una dirección.
`allowance(titular, autorizado)`	Consulta	Monto que el autorizado aún puede gastar.
`transfer(destino, monto)`	Transacción	Mueve tokens desde quien firma; revierte si el saldo no alcanza.
`approve(autorizado, monto)`	Transacción	Autoriza a otra cuenta a gastar hasta el monto indicado.

Función	Tipo	Descripción
`transferFrom(origen, destino, monto)`	Transacción	Mueve tokens dentro del límite autorizado y lo reduce.

El contrato emite los eventos `Transfer(origen, destino, monto)` y `Approval(titular, autorizado, monto)`. En la emisión inicial, el origen de `Transfer` es la dirección cero.

Funciones excluidas

Ninguna exclusión obedece a una limitación técnica: OpenZeppelin ofrece módulos para varias de ellas [2]. Se excluyen porque cada una añadiría privilegios, riesgos o connotaciones económicas que no aportan al objetivo.

Función excluida	Qué permitiría	Por qué no se incluye
Emisión posterior (mint)	Crear tokens después del despliegue.	Rompería el suministro fijo y exigiría una cuenta con permiso de emisión, cuya clave sería un punto único de falla.
Quema (burn)	Destruir tokens.	La administrativa altera saldos de terceros y contradice la autocustodia; la voluntaria modifica `totalSupply` sin aportar al objetivo.
Impuestos o comisiones	Descontar un porcentaje en cada transferencia.	El receptor no recibiría el monto exacto, lo que rompe el criterio de prueba y la compatibilidad con aplicaciones estándar.
Pausas	Detener todas las transferencias.	Exige un administrador con poder sobre todos los saldos; en un grupo cerrado no hay emergencia que lo justifique.
Listas de bloqueo	Impedir que ciertas direcciones envíen o reciban.	Requiere un administrador y criterios que no tienen sentido entre cuatro integrantes.

Función excluida	Qué permitiría	Por qué no se incluye
Proxy actualizable	Reemplazar la lógica después del despliegue.	Eliminaría la inmutabilidad que se quiere observar y añade complejidad y una clave de administración.
Venta pública, preventa o recompensas	Intercambiar B4T por POL u otros activos, o repartirlo como incentivo.	Contradice la finalidad educativa y tendría implicaciones económicas y regulatorias.

Por qué el suministro es fijo

1. En OpenZeppelin Contracts 5.x, `_mint` es una función `internal`: solo el propio contrato o sus herederos pueden invocarla, nunca una cuenta externa [2].
2. El constructor se ejecuta una sola vez, en la transacción de despliegue. Si `_mint` solo se llama allí, después no existe ningún camino de ejecución que aumente el suministro.
3. OpenZeppelin presenta este patrón como la forma recomendada de crear un token de suministro fijo [6].

Cualquier persona puede confirmarlo revisando el código verificado en Polygonscan y comparando `totalSupply()` antes y después de las pruebas. Sin funciones administrativas tampoco existe una cuenta privilegiada cuya clave pueda robarse; las guías de seguridad de Ethereum recomiendan contratos simples y bibliotecas auditadas [7].

5. Plan de pruebas y criterios de despliegue

El contrato solo se despliega en Polygon PoS después de aprobar los 18 casos de Amoy y la revisión por pares. Amoy cubre también los casos límite y de error, porque una transacción revertida consume gas real y no se repite en mainnet.

Pruebas en Amoy (A01 a A18)

ID	Caso	Criterio de aprobación
A01	Compilación	Sin errores ni advertencias críticas en el análisis estático de Remix [4], [5].

ID	Caso	Criterio de aprobación
A02	Despliegue y emisión	10^24 unidades mínimas en la billetera de pruebas y evento `Transfer` desde la dirección cero.
A03	Metadatos	`name`, `symbol` y `decimals` devuelven "Block4 Token", "B4T" y 18.
A04	Distribución inicial	Cada integrante recibe exactamente 250 000 B4T (4 hashes).
A05	Transferencias entre pares	12 hashes, uno por par ordenado, con montos enteros y fraccionarios; el receptor recibe el monto exacto.
A06	Monto mínimo	Transferir 1 unidad mínima aumenta en 1 el saldo del receptor.
A07	Saldo completo	El saldo del emisor queda en cero.
A08	Monto cero	Se acepta y emite `Transfer`, como exige el estándar [1].
A09	Autotransferencia	El saldo no cambia y se emite `Transfer`.
A10	Saldo insuficiente	Reversión con `ERC20InsufficientBalance`; sin cambio de saldo ni evento.
A11	Dirección cero	Reversión con `ERC20InvalidReceiver`.
A12	Autorización	Evento `Approval` y `allowance` igual al monto aprobado.
A13	Gasto delegado	`Transfer` emitido y `allowance` reducido en el monto gastado.
A14	Exceso de autorización	Reversión con `ERC20InsufficientAllowance`.
A15	Cambio de autorización	Llevar el `allowance` a cero y asignar un nuevo valor; el valor final es el nuevo.
A16	Ausencia de funciones privilegiadas	El ABI y el código verificado no tienen funciones públicas de mint, burn, pausa ni administración.
A17	Conservación	La suma de saldos de las cuentas involucradas coincide con `totalSupply`.

ID	Caso	Criterio de aprobación
A18	Verificación	Contrato verificado en el explorador de Amoy; código fuente y compilador coinciden.

Criterios para autorizar el despliegue en Polygon PoS

1. Los casos A01 a A18 están aprobados, con hashes y capturas como evidencia.
2. La conciliación de Amoy cuadra con `totalSupply` y el contrato está verificado.
3. Al menos dos integrantes distintos de quien preparó cada elemento revisan el código, los parámetros, los resultados y el presupuesto, y firman la lista de verificación.
4. El presupuesto de gas está aprobado por el equipo, calculado con el consumo medido en Amoy más un margen.
5. El código y la configuración del compilador son los mismos que se probaron en Amoy.
6. No hay frases semilla, claves ni datos personales en el repositorio ni en las capturas.

Pruebas en Polygon PoS (M01 a M07)

En mainnet se ejecutan solo operaciones válidas: al menos 18 transacciones confirmadas, a saber, 4 de distribución, 12 entre pares (4 × 3 pares ordenados) y 2 de delegación.

ID	Caso	Criterio de aprobación
M01	Despliegue	Desde la billetera exclusiva del proyecto, en chain ID 137; se registran dirección, hash y gas.
M02	Verificación	Código publicado en Polygonscan, con compilador y parámetros idénticos a Amoy.
M03	Distribución inicial	4 hashes; cada integrante con 250 000 B4T y la billetera del proyecto con 0.
M04	Transferencias entre pares	12 hashes, uno por par ordenado, con montos exactos.
M05	Delegación	1 `approve` y 1 `transferFrom`; eventos `Approval` y `Transfer`; `allowance` actualizado.
M06	Conciliación final	La suma de saldos es igual a `totalSupply`.
M07	Costos	La suma del gas pagado está dentro del presupuesto aprobado.

6. Billetera exclusiva del proyecto

El despliegue y la distribución inicial se firman desde una billetera creada solo para el proyecto, nunca desde la de un integrante. Así el historial público de la cuenta contiene únicamente operaciones del experimento, no se exponen fondos ni actividad personal y cualquiera puede auditar qué hizo.

1. **Creación.** Se genera una billetera nueva, nunca usada, en un equipo de confianza. Su dirección pública se registra en el informe desde el inicio.
2. **Custodia.** La frase semilla se escribe solo en papel. Un integrante designado es el custodio principal y un segundo conserva un respaldo sellado. Nunca se fotografía, digitaliza ni comparte por chat o correo.
3. **Fondeo.** Recibe únicamente el POL del presupuesto de gas aprobado por el equipo, calculado con el consumo medido en Amoy más un margen. La transacción de fondeo se registra.
4. **Uso exclusivo.** Solo firma el despliegue y las cuatro transferencias de distribución. No se conecta a otras aplicaciones ni recibe otros activos.
5. **Verificación previa a cada firma.** Antes de firmar se confirma en la lista de verificación la red (chain ID 137), la versión del compilador, los parámetros del constructor, el saldo de POL y las direcciones de destino. La firma se hace en una sesión conjunta en la que al menos dos integrantes revisan la pantalla de confirmación.
6. **Cierre.** Al terminar la distribución, la billetera queda con saldo cero de B4T. El POL sobrante se documenta y la billetera se retira: no se reutiliza.

Para Amoy se usa una billetera de pruebas separada de las personales, fondeada con POL de faucet. No se usa una billetera multifirma porque el contrato no tiene funciones administrativas: la billetera del proyecto solo es relevante durante el despliegue y la distribución, y después no conserva ningún privilegio sobre el token.

7. Riesgos y medidas de mitigación

Aunque el contrato es mínimo, el experimento usa claves reales en una red pública, y una transacción confirmada no se puede revertir ni un contrato desplegado se puede corregir. Por eso cada riesgo tiene una medida que se verifica antes de firmar.

Riesgo	Descripción	Medida adoptada	Ref.
Exposición o pérdida de claves	Quien conozca la clave o la frase semilla controla la billetera; si se pierden, el acceso es irrecuperable.	Frases semilla solo en papel, nunca en repositorio, chat, nube ni capturas; billetera exclusiva para el despliegue con custodia definida (sección 6).	[7]
Errores en el código	Un contrato desplegado no se puede corregir.	Herencia de OpenZeppelin sin lógica propia, compilador fijo, análisis estático de Remix, matriz completa en Amoy y revisión cruzada.	[2], [7]
Funciones privilegiadas	Un rol de administrador concentra el riesgo en una sola clave.	Eliminado: el contrato no tiene funciones administrativas.	[7]
Condición de carrera en `approve`	Al cambiar una autorización, el autorizado podría gastar el valor anterior y el nuevo.	Llevar la autorización a cero antes de asignar un nuevo valor; se prueba en Amoy (A15).	[1]
Envío a dirección equivocada	ERC-20 no confirma que el receptor exista; un envío errado se pierde.	Libreta de direcciones validada por cada titular y transferencia de prueba de bajo monto antes de la distribución. El contrato rechaza la dirección cero.	[2]
Red equivocada	Desplegar en mainnet creyendo que es Amoy, o al revés.	Verificar el chain ID (137 o 80002) en Remix y en la billetera antes de firmar, con lista de verificación.	[3], [9]
Versión del compilador	Usar otra versión o configuración impide verificar el código.	Versión estable compatible con OpenZeppelin, fija en el código; se registran versión y optimización.	[8]

Riesgo	Descripción	Medida adoptada	Ref.
Phishing y firmas maliciosas	Sitios falsos de Remix, faucets o dApps pueden pedir firmas que vacían la billetera.	Solo URLs oficiales, leer cada solicitud de firma y no conectar la billetera del proyecto a otras aplicaciones.	[7]
Costos superiores a lo previsto	El precio del gas varía y las transacciones revertidas también lo consumen.	Medir el gas en Amoy, aprobar un presupuesto con margen, fondear solo ese monto y probar los errores solo en Amoy.	[3]
Interpretación como activo financiero	Terceros podrían percibir el token como inversión.	Declaración expresa de uso educativo, sin venta, listado ni liquidez (sección 8).	—

8. Declaración de uso educativo y reglas de conducta

B4T es un instrumento de aprendizaje sin valor prometido. No se promoverá, venderá ni listará en mercados, no ofrece rentabilidad y no sirve como medio de pago. Circula únicamente entre las billeteras de los cuatro integrantes del equipo.

Reglas de conducta del grupo

- B4T no se transfiere a terceros ni se ofrece a nadie fuera del equipo; no se crea liquidez ni se intercambia por POL u otros activos.
- Cada integrante firma personalmente desde su propia billetera y conserva sus claves privadas.
- Las frases semilla y las claves privadas nunca se escriben en el repositorio, el chat, la nube ni las capturas. Si algo se filtra, se avisa de inmediato al equipo y se reemplaza la billetera afectada.
- Ninguna transacción en mainnet se firma sin que al menos dos integrantes hayan revisado sus parámetros.
- Los cambios al contrato entran por pull request con la aprobación de al menos dos integrantes distintos del autor. El commit desplegado en cada red se marca con un tag.

- La evidencia se limita a direcciones públicas y hashes de transacción. Las capturas se revisan antes de subirlas y no muestran saldos de otros activos ni datos personales.
- Los casos de error se ejecutan solo en Amoy.
- Las decisiones técnicas, las revisiones y las evidencias son responsabilidad del equipo completo; cada integrante debe poder explicar las pruebas realizadas.

Anexo. Registro de despliegue

Estos datos se completan con información pública después de cada despliegue; hasta entonces permanecen pendientes y el documento no anticipa resultados.

Dato	Polygon Amoy (80002)	Polygon PoS (137)
Dirección de la billetera que despliega	Pendiente	Pendiente
Dirección del contrato	Pendiente	Pendiente
Hash de despliegue	Pendiente	Pendiente
Gas consumido en el despliegue	Pendiente	Pendiente
Contrato verificado en el explorador	Pendiente	Pendiente
Tag del commit desplegado	`v1.0-amoy`	`v1.0-mainnet`
Costo total de las pruebas (POL)	Pendiente	Pendiente

Referencias

[1] F. Vogelsteller y V. Buterin, "ERC-20 Token Standard", Ethereum Improvement Proposal 20, nov. 2015. [En línea]. Disponible en: eips.ethereum.org/EIPS/eip-20

[2] OpenZeppelin, "ERC 20", OpenZeppelin Contracts Documentation, 2026. [En línea]. Disponible en: docs.openzeppelin.com/contracts/5.x/erc20

[3] Polygon Labs, "Building on Polygon PoS", Polygon Developer Docs, 2026. [En línea]. Disponible en: docs.polygon.technology/pos/get-started/building-on-polygon

[4] Remix Project, "Remix Documentation", 2026. [En línea]. Disponible en: remix-ide.readthedocs.io

[5] Polygon Labs, "Remix", Polygon Developer Docs, 2026. [En línea]. Disponible en: docs.polygon.technology/tools/dApp-development/common-tools/remix

[6] OpenZeppelin, "Creating ERC20 Supply", OpenZeppelin Contracts Documentation, 2026. [En línea]. Disponible en: docs.openzeppelin.com/contracts/5.x/erc20-supply

[7] Ethereum Foundation, "Smart contract security", ethereum.org Developer Docs, 2026. [En línea]. Disponible en: ethereum.org/en/developers/docs/smart-contracts/security

[8] Solidity Team, "Security Considerations", Solidity Documentation, 2026. [En línea]. Disponible en: docs.soliditylang.org/en/latest/security-considerations.html

[9] V. Buterin, "EIP-155: Simple replay attack protection", Ethereum Improvement Proposal 155, oct. 2016. [En línea]. Disponible en: eips.ethereum.org/EIPS/eip-155